16 貪婪演算法

Part IV Advanced Design and Analysis Techniques

本章定位

貪婪演算法每步只保留一個選擇，節省枚舉其他候選的成本。本章用活動交換、Huffman葉交換與matroid增廣性證明這一步何時安全，再處理加權目標與容量限制如何使原規則失效。

前置知識

排序、交換論證、樹結構與選擇公理性質。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 用交換法證明 activity selection。
- 辨識 greedy-choice 與最佳子結構。
- 建 Huffman code 並以 matroid 理解貪婪正確性。
- 在安全選擇失效時給出反例，並把一般證明中的交換步驟寫完整。
- 16.1 An activity-selection problem
- 16.2 Elements of the greedy strategy
- 16.3 Huffman codes
- 16.4 Matroids and greedy methods
- 16.5 A task-scheduling problem as a matroid

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16.1 An activity-selection problem

已知所有活動的開始、結束時間，要求選最多個兩兩不重疊活動。假設 s_i<f_i，活動已按 finish time 遞增排序，區間採半開形式 `[s_i,f_i)`，故一個活動開始時刻等於另一個結束時刻也相容。

定理。 對任一非空子問題，令 `a_m` 是 finish 最早的活動，則存在最大相容集合包含 `a_m`。

證明。 令 A^* 為子問題的最大解，a_j 為 A^* 中最早結束的活動。因 a_m 是全域最早結束，`f_m<=f_j`。把 a_j 換成 a_m：

- A^* 中原本在 a_j 之後的活動都滿足 `start>=f_j`，所以也與 a_m 相容；
- 子問題沒有更早的 activity 可受影響；
- 集合大小不減，故替換後仍是最大解。

因此 greedy-choice 成立。選 a_m 後只剩 `start>=f_m` 的子問題；對剩餘集合重複同一交換，歸納得到最優。

活動選擇的線性掃描與邊界

```
activity-select(按結束時間排序的活動):
    result=[]; last_finish=-∞
    for (s,f) in 活動:
        if s>=last_finish:
            append (s,f) to result
            last_finish=f
    return result
```

每次處理活動 j 前，result 是前 j−1 個中依此規則取出的相容集合，last_finish 是最後所選活動的終點；交換定理使每次選擇可延伸為某個最優解。掃描每活動一次，已排序時 Θ(n)；若尚未排序，另付 O(n log n)。

本章假設 s<f 且區間半開，端點相等可以相接；零長活動需另訂同點使用規則。空輸入直接輸出空集合。最早開始或最短時長不保證最優，例如活動[0,10)、[1,2)、[2,3)使最早開始只選一個，卻有兩個相容短活動。

最短時長先取的反例為 [0,3)、[3,6)、[2,4)。先選長度2的中間活動會擋住兩個長度3的活動，只有一個；最優則選前兩個，得到兩個。

活動選擇的完整掃描

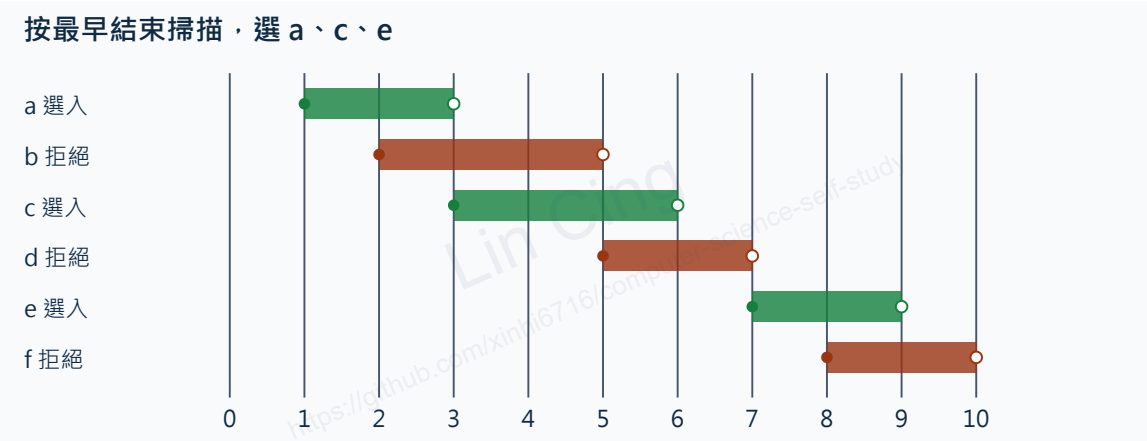
取六個活動：

活動	start	finish
a	1	3
b	2	5
c	3	6
d	5	7
e	7	9
f	8	10

依 finish 排序掃描：選 a；b 的 $2<3$ 拒絕；c 的 $3>=3$ 選入；d 的 $5<6$ 拒絕；e 的 $7>=6$ 選入；f 的 $8<9$ 拒絕。結果 `{a,c,e}`。

活動選擇的完整掃描 （續）

最優解也不只一個。`{b,d,e}` 與 `{a,c,e}` 都有 3 個；交換定理只保證存在一個含最早結束者的最優解，沒有說所有最優解都必須包含它。



16.2 Elements of the greedy strategy

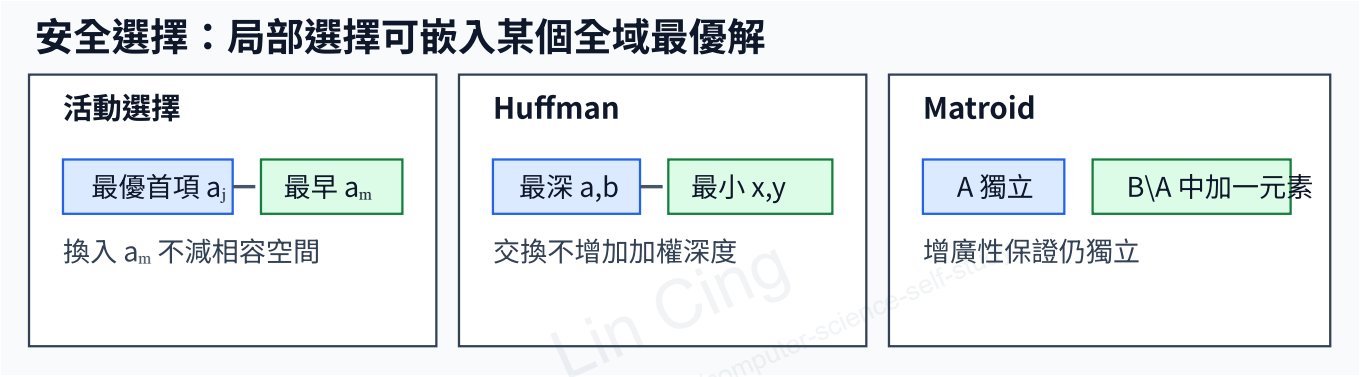
本章的交換型 greedy proof 要回答下列問題：

1. **Greedy choice 安全。** 取 greedy 的 g，對任意最優解 A^* 找第一個不同決策 x，構造 $A' = A^* - x + g$；逐項驗證 A' 可行且價值不低於 A^*。
2. **剩餘問題相同。** 移除 g 後，A^* 的其餘部分必須是明確定義的子問題的可行解，且子問題沒有遺失原來的候選。
3. **歸納終止。** 基底（空集合或單一活動）正確，每次決策使問題嚴格縮小。

這三項成立才可由「每步都安全」推到全域最優。局部立即利益、平均表現或若干成功例都不足夠。

例如任意面額的找零不能只取最大硬幣。面額 {1,3,4} 找6，greedy 得4+1+1三枚，最佳3+3兩枚；此反例否定一般規則。熟悉面額的成功經驗不能代替針對面額族的證明。

16.2 Elements of the greedy strategy （續）



為何加權活動不能用同一交換

把目標改成最大總 value，就不再是最大基數。取

活動	區間	value
short	[0,2)	1
long	[1,3)	100
tail	[3,4)	1

最早 finish greedy 選 short，再選 tail，總 value 2；最優為 long+tail，總 value 101。交換 A^* 中原本最早結束的 long 為 short 後，雖然區間仍相容，但 value 從 100 降到 1，不能保持最優值。

加權版本有最佳子結構。以 1-based 索引按 finish 排序，令 p(j) 為 j 之前 finish 不晚於 s_j 的最大索引，不存在時 p(j)=0；D[j] 表示前 j 個活動的最佳總價值。則 `D[0]=0`，

`D[j]=max(D[j-1], value_j+D[p(j)])`。

為何加權活動不能用同一交換 （續）

排除 j 就只看前 j−1 個；包含 j 則其餘活動必來自前 p(j) 個，而且可獨立最佳化。此二分涵蓋所有解。數例 p=(0,0,2)、D=(0,1,100,101)，由 D3 選 tail，再由 D2 選 long。finish-time greedy 只保證未加權最大基數，不能沿用到這個目標。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

0–1 與 Fractional Knapsack 的不同證明

假設物品重量與價值皆正、容量 C>0。fractional 版本允許取比例 x_i∈[0,1]，其餘目標是最大化 Σv_i x_i。依密度 ρ_i=v_i/w_i 下降排序，先裝滿密度高的物品，至多一件只裝部分。

若全部物品總重不超過 C，全部取走即最優。否則任何最優解必裝滿 C；若仍有空間且有正價值物品未取完，增加一點比例就能改善目標，矛盾。

若某可行最優解在高密度 i 尚未取滿時，卻使用較低密度 j 的正重量，令 δ=min(w_i(1−x_i),w_j x_j)>0，把 δ 重量由 j 移給 i。容量不變，價值增加 δ(ρ_i−ρ_j)≥0。反覆交換直到第一個 greedy 選擇相同，再對剩餘容量歸納，得到某最優解符合整個 greedy 排序。相同密度可任意打破平手。

但在 0–1 版本 x_i 只能是 0 或 1，δ 重量交換通常不合法。例 C=10，物品 (w,v)=(6,12),(5,9),(5,9)。密度 greedy 先取 (6,12)，剩 4 放不下其他，價值 12；0–1 最優取兩件 (5,9)，價值 18。fractional 則先取 (6,12)，再取一件 (5,9) 的 4/5，價值 19.2。

0–1 DP 用 D[i,c] 表示前 i 件、容量至多 c 的最佳值，D[0,c]=D[i,0]=0，D[i,c]=max(D[i−1,c],v_i+D[i−1,c−w_i])，第二項僅在 w_i≤c 時存在。整數容量的時間 Θ(nC) 是偽多項式，因輸入 C 本身只需 Θ(log C) 位元。

16.3 Huffman codes

二元 prefix tree 的葉深度就是碼長。給定頻率 `f_c`，總位元數為

`B(T)=sum_c f_c d_T(c)`。

Huffman 每次從 priority queue 取兩個最小權重樹 x,y，建立權重為 `f_x+f_y` 的父節點。若有 n 個符號，進行 n−1 輪；每輪兩次 extract-min、一次 insert，共 3(n−1) 次 heap 操作，加線性建堆，時間 O(n log n)。

交換左、右 child 只改變 codeword 的 0/1，不改變長度與總成本。頻率相同時可形成多棵不同的最優樹。

Huffman 選擇引理：把最小頻率放到最深兄弟

假設至少兩個字元且頻率皆正。任何最佳 prefix tree 可取成每個內節點恰有兩個 child，因刪掉只有一個 child 的內節點會縮短其下所有碼字而降低成本。取最深葉，其兄弟也必為同深葉，否則兄弟的後代會更深。

令 x、y 為頻率最低的兩字元。先把 x 換到這對最深葉的一個位置；若其中一個位置原本是 y，就選另一位置。這使 y 在第二次交換前仍留在別處，或已經在所需的兄弟位置。

交換 a 與 x 的成本差只涉及這兩個葉：

`B(T')-B(T)=(f_a-f_x)(d_x-d_a)<=0`。

因 `f_a>=f_x` 且 `d_x<=d_a`，差值非正。此時 x 已放妥，剩餘位置的頻率都不小於 y，故再把 y 換到 x 的最深兄弟位置，同樣不增加成本，且不會移走 x。若 x 或 y 原本已在目標位置，該次交換為零。頻率相同時任何相應安排成本相等。

故總存在最佳 prefix code 含這個 greedy choice。這裡需使用「最低頻率」與「最深葉」的兩個交換不等式，不能只說看起來合理。

Huffman 收縮與最優子結構

把 x、y 從字母表刪除，加入虛擬字元 z，令 `f_z=f_x+f_y`。若 T' 是收縮後的最佳樹，把 z 展開成 x、y，得到原樹 T。

除 x、y 以外的葉深度不變；x、y 的深度都是 `d_T'(z)+1`，故

`B(T)=B(T')+f_x+f_y`。

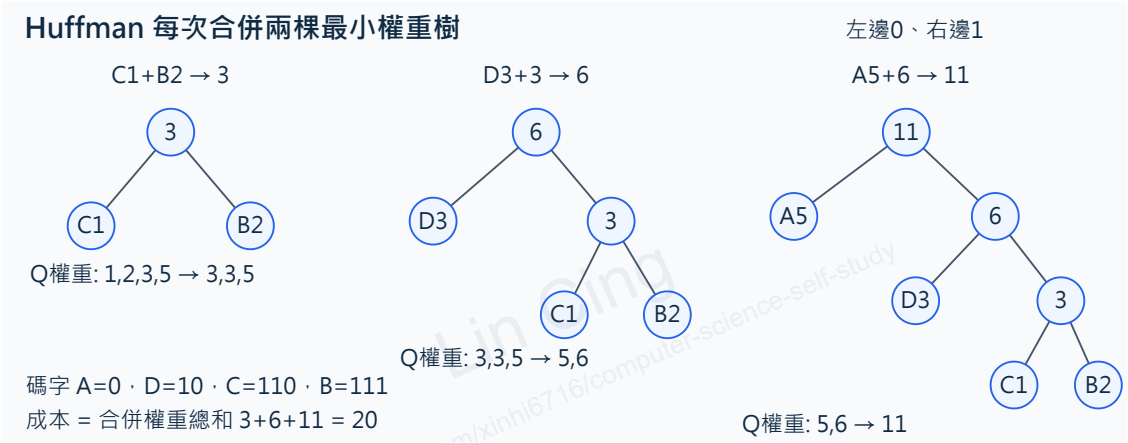
若原問題有更優樹 T*，依上一頁可令 x、y 為最深兄弟；將兄弟收縮成 z，得到

`B(T**)=B(T*)-f_x-f_y<B(T')`，

與 T' 最優矛盾。因此展開收縮樹即原問題最優；再以符號數歸納，Huffman 正確。

頻率 A5、B2、C1、D3 的合併權重為 3、6、11；可取 A=0、D=10、C=110、B=111，成本 `5+6+3+6=20=3+6+11`。

Huffman 收縮與最優子結構 （續）



Huffman 的演算法與單字元邊界

```
Q = 所有正頻率單葉樹組成的 min-heap
while size(Q)>1:
    x=extract_min(Q); y=extract_min(Q)
    insert(Q, node(x.weight+y.weight, x, y))
return Q 的唯一樹 · 若 Q 空則回傳空
```

解碼從根依0、1走向左、右 child；到葉就輸出字元並重回根。prefix 性質保證沒有某個完整碼字同時是更長碼字的前綴，因而字界明確。最後位元若停在內節點，表示輸入截斷，不能默認成功解碼。

空字母表沒有碼樹。只有一個字元時，樹論可給空碼，實際串流通常約定碼0並另傳字元數。本章成本與合併權重總和的等式採至少兩個字元的完整二元樹，不能把單字元實作約定偷換成合併成本。

16.4 Matroids and greedy methods

matroid `M=(S,I)` 有三條：

- 空集屬於 I；
- 遺傳性：若 A 屬於 I，任何 A 的子集也屬於 I；
- 增廣性：若 A,B 屬於 I 且 |A|<|B|，存在 `x in B\A` 使 `A∪{x}` 屬於 I。

假設 S 有限，w(x)>0。greedy 依 weight 降冪掃描，加入後仍獨立就取。令 x 是第一個單點獨立的元素。若不存在，遺傳性使所有非空集合都不獨立，空集就是最優解。

取任一最優集 B。若 x∉B，從 A={x} 開始，當 |A|<|B| 就以增廣性選 b∈B\A 加入 A。每次只從 B 加入，最後 |A|=|B|，故恰有一個 y∈B 被省去，A=B−{y}+{x}。由 B 獨立可知 {y} 獨立，x 是權重最高的可行單點，故 w(x)≥w(y)，而 w(A)=w(B)−w(y)+w(x)≥w(B)。因此 A 也最優且含 x。

這裡交換性不是直接宣告「換任一 y 都可行」。必須先以逐步增廣構造出可行的 A，再由等大小推得唯一被省去的 y。

收縮後仍為 Matroid 的完整證明

固定可行的 x，令 S'=S\{x}，I'={A⊆S' : A∪{x}∈I}。S' 可以包含在新系統中永不獨立的元素，這些元素不影響結論。

- 空集在 I'，因 {x}∈I。
- 若 A∈I'、C⊆A，則 C∪{x}⊆A∪{x}，由原系統遺傳性可得 C∈I'。
- 若 A,B∈I' 且 |A|<|B|，原系統中的 A∪{x}、B∪{x} 都獨立且前者較小。增廣元素 y 必在 `(B∪{x})\(A∪{x})=B\A`，因此 A∪{y}∈I'。

對任何含 x 的可行解，其餘部分屬 I'，且權重恰少 w(x)；反之亦然。所以原問題在選定 x 後精確化為收縮系統的最大權獨立集。對 |S| 歸納，後續掃描的可行性正是 I' 的 oracle，greedy 全程最優。

若 x 在某階段不可加入，之後也不會可加入。否則較大集合 A∪{x} 獨立會由遺傳性推出當時較小的集合也可加 x，矛盾。排序 O(n log n)，每次 oracle 成本 O(f(n)) 時，總成本 O(n log n+n f(n))。允許負權且目標為任意獨立集時，須略過負權元素；不能無條件取成最大基底。

Graphic Matroid 與非 matroid 反例

對固定無向圖 G=(V,E)，令 S=E，I 為所有不含 cycle 的邊集合，孤立頂點仍列入 V。遺傳性成立。令 A 的分量為 V1..Vk；若 B 每條邊都留在某個 Vi 內，因 B 在 Vi 內仍是森林，邊數至多 |Vi|−1。相加得 |B|≤Σ(|Vi|−1)=|V|−k=|A|，與 |B|>|A| 矛盾。所以存在 B 中一條跨 A 分量的邊，加入 A 不形成 cycle，增廣性成立。

連通圖的基底都是 n−1 邊的生成樹。若要由最大正權定理推得最小成本 MST，可令新權重 w'(e)=K−cost(e)>0；所有基底有相同邊數，故最大化 Σw' 等價於最小化原成本。這使 Kruskal 升冪規則與此定理精確相接，並沒有把最大任意森林與最小任意森林混為一談。

但一個 hereditary system 不一定有增廣性。取 S={a,b,c}、權重 10,9,9，I={∅,{a},{b},{c},{b,c}}。A={a}、B={b,c} 有 |A|<|B|，卻無法把 b 或 c 加入 A；這不是 matroid。Greedy 選 a 得 10，最優 {b,c} 得 18。

16.5 A task-scheduling problem as a matroid

工作 j 處理時間 1、deadline d_j、延遲罰金 w_j。集合 A 可全部準時，當且僅當依 deadline 排好後 `d_(k)>=k`；等價於對每個 t，

`N_t(A)=|{j in A:d_j<=t}|<=t`。

必要性是前 t 個時間槽最多容納 t 個工作。充分性是把 A 按 deadline 遞增排；第 k 個工作的 deadline 至少 k，故不遲到。

所有工作在時間 0 就緒、不可搶占、單機，deadline 是正整數，完成於 d_j 視為準時。n 個工作只需前 n 格，故先將大於 n 的 deadline 截為 n，不改變可行性。

這族獨立集有遺傳性。為證增廣，取 |A|<|B|，令 k 是滿足 N_k(A)=k 的最大整數，0 一定符合。由 |A|<|B|≤n，k<n。A 中有 k 個工作 deadline≤k，而 B 這類工作至多 k 個，所以 B 中 deadline>k 的工作數至少 |B|−k>|A|−k。因此存在 x∈B\A 且 d_x>k。

把 x 加入 A。對 t≤k，新增工作不被計入，所以 N_t 不變；對 t>k，最大性保證 N_t(A)<t，整數性使 N_t(A)≤t−1，新增至多 1 後仍≤t。因此 A∪{x} 可行，增廣性成立。

單位時間排程的 greedy 與完整追蹤

工作：

工作	deadline	penalty
a	3	9
b	1	8
c	3	7
d	2	6
e	1	5

按 penalty 由大到小加入：a 可行；b 使 deadlines 排序 1,3，可行；c 使 1,3,3，可行；d 使 1,2,3,3，第 4 個 deadline 3<4，拒絕；e 使 `N_1=2>1`，拒絕。

準時集為 {a,b,c}，依 deadline 排成 b,a,c，分別於 1、2、3 完成。其餘 d,e 放在 4、5，確實遲到。延遲罰金 6+5=11；總 penalty 35，準時工作所保住的 penalty 為 24，達最大。

單位時間排程的 greedy 與完整追蹤 （續）

使用長度 n 的 deadline 次數陣列，計算各前綴 N_t，可在 O(n) 內檢查一次獨立性，n 次檢查共 $O(n^2)$，加最初 penalty 排序 O(n log n)。若每次另作比較排序，oracle 是 O(n log n)，總時間會變成 $O(n^2 \log n)$，不能把排序叫作線性。

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 16。

Cormen et al., *Introduction to Algorithms*, 3rd ed., § §16.1-16.5.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study